# Predict-film-category: Project Summary

Tesfabirhan REDIE 28/10/2026

*Horror vs. comedy film classification, from notebook to deployed API*

## 1. Objective

The goal of this project is to predict whether a film is a **horror** film or a **comedy** from its plot summary alone. It is a binary text-classification problem that starts as an exploratory NLP notebook and ends as a containerized, tested and versioned web service running on Azure. The project is run by a single person over five working days (22 to 28 October 2026) and is meant to demonstrate the full path from model to production, not only model accuracy.

## 2. Data

The dataset is `textes_films_horreur_comedie.csv`, with 2,400 rows and three columns:

- `titre`: the film title
- `resume`: the plot summary (the model input)
- `cible`: the label, 1 for horror and 0 for comedy

The summaries are free text, so the work focuses on cleaning and vectorizing language before any model is trained.

## 3. Modelling approach

The modelling work is organized as an eight-stage notebook (`predFilm_clean.ipynb`), built on the earlier NLP exercises (NLTK stemming, spaCy lemmas and entities, TF-IDF with logistic regression).

1. **Cleaning**: normalize the text and remove noise and stop words.
2. **Split**: 60% train, 20% validation, 20% test, so that tuning and final evaluation stay separate.
3. **TF-IDF vectorization**: turn each summary into a weighted bag-of-words representation.
4. **Baselines**: simple reference models, notably TF-IDF with logistic regression, that any more complex model must beat.
5. **KerasTuner**: hyperparameter search for a Keras neural network.
6. **Optuna**: a second, independent search to compare tuning strategies.
7. **Verdict**: compare all candidates on the held-out data and choose one, with the reasoning written down.
8. **Save**: persist the chosen model and its vectorizer so the API can load them unchanged.

The key design choice is to compare a neural network against strong linear baselines honestly. On short-text TF-IDF problems, a regularized logistic regression is often hard to beat, so the verdict stage matters as much as the tuning.

## 4. From model to online application

The second half of the brief turns the saved model into a service:

- **FastAPI** exposes a prediction endpoint that takes a summary and returns the predicted category.
- **pytest** covers the API and the prediction logic.
- **Quality gate**: a minimum bar (tests passing, and a performance threshold) that a build must clear before release.
- **Docker** packages the application and model into one reproducible image.
- **GitHub Actions** runs tests and builds the image on every push.
- **Azure**: the image is pushed to Azure Container Registry (registry `tesfcp`) and served on Azure Container Apps (environment `film-env`).
- **Redeploy and rollback**: each release is versioned so a bad version can be replaced by a previous one quickly.
- **Cleanup**: the cloud resources are removed at the end of the project.

## 5. Project management and documentation

The project was planned for one person over five working days, with the weekend skipped, using a Notion project-management template and exported as Markdown. Three documents live in the repository next to the code:

- `PLAN.md`: a task table with dependencies, estimated and actual effort, resources and planned dates.
- `PROCEDURE_MISE_EN_PRODUCTION.md`: the step-by-step release procedure, with a version log.
- `FIN_DE_VIE.md`: how the service is decommissioned and its resources cleaned up.

Tracking estimated against actual effort is deliberate: it makes the plan reviewable and shows where the junior-level estimates were off.

## 6. Results

*To complete with the final figures from the notebook:*

- Test accuracy and F1: **Test F1: 0.848 ± 0.032 (n = 469)**

| | model | mean F1 | std | noise margin |
|---|---|---|---|---|
| 0 | Logistic regression | 0.842 | 0.018 | 0.043 |
| 1 | Naive Bayes | 0.859 | 0.023 | 0.041 |
| 2 | Baseline network | 0.84 | 0.019 | 0.043 |
| 3 | Tuned network | 0.846 | 0.02 | 0.042 |

- Best hyperparameters found by KerasTuner and by Optuna

```
Optuna: F1 = 0.855 | {'n_layers': 3, 'units': 224, 'dropout': 0.0, 'l2': 2.7155819552829413e-05, 'lr': 0.0001705053926026929}
```

- Latency and size of the deployed container

```
Trial 20 Complete [00h 00m 25s]
val_f1: 0.8405797101449275

Best val_f1 So Far: 0.8582089552238806
Total elapsed time: 00h 06m 42s
KerasTuner: F1 = 0.858 | {'n_layers': 2, 'units': 32, 'dropout': 0.2, 'l2': 1.2232117875737861e-05, 'lr': 0.005977728042983696}
```

## 7. Skills demonstrated

Text preprocessing and TF-IDF features, model comparison with a held-out test set, hyperparameter optimisation (KerasTuner, Optuna), API design with FastAPI, automated testing, containerisation, CI/CD with GitHub Actions, deployment on Azure, and basic project planning and operational documentation.

## 8. Limitations and next steps

- Only two classes; real catalogues have many overlapping genres, so a multi-label extension would be natural.
- Bag-of-words ignores word order and context; pretrained embeddings or a transformer would likely capture tone better.
- Plot summaries are short and stylistically uneven, so more data and cross-validation would give more reliable estimates.